

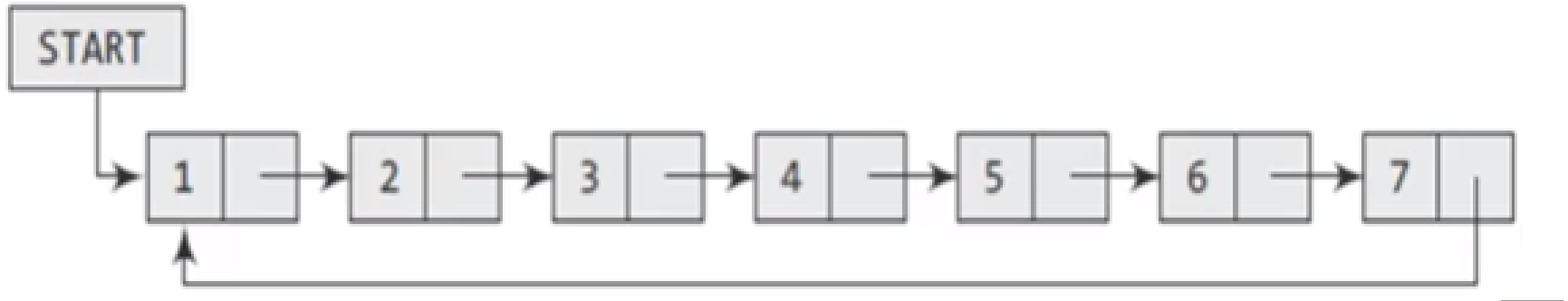
# **Circular & Doubly Linked List**

# Circular Linked List

# CIRCULAR LINKED LIST

In a circular linked list, the **last node contains a pointer to the first node** of the list.

While traversing a circular linked list, we can begin and traverse the list in forward direction until we reach the same node where we started.



# Inserting a New Node in a Circular Linked List

**Case 1:** The new node is inserted at the beginning of the circular linked list.

**Case 2:** The new node is inserted at the end of the circular linked list.

**Case 3:** insertion somewhere in between the two nodes.

# Insertion at the Beginning

Steps to insert a Node at beginning :

1. The first Node is the Head for any Linked List.
2. When a new Linked List is instantiated, it just has the Head, which is Null.
3. Else, the Head holds the pointer to the first Node of the List.
4. When we want to add any Node at the front,
  - we must make the **head point to newly added node**.
  - **Next pointer of newly added Node, must point to previous Head**, whether it be NULL(in case of new List) or the **pointer to the first Node of the List**.
  - The previous Node pointed by Head is now the second Node of Linked List, because the new Node is added at the front.

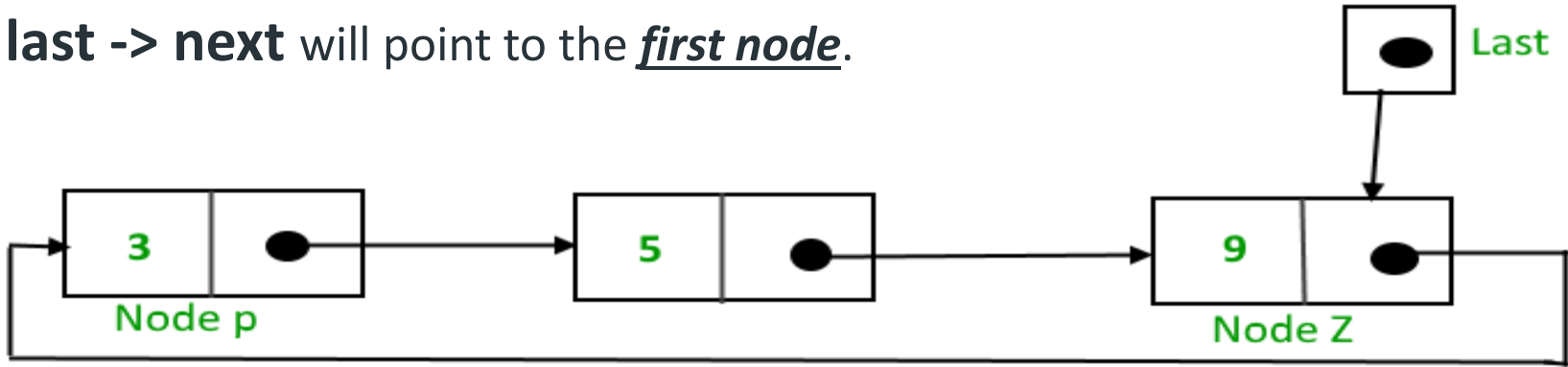
```
class Node
{
    int info;
    Node *next;
};
```

```
class circular_list
{
    public:
    Node *last;
    circular_list()
        {last = NULL;}
};
```

## To implement a circular singly linked list

Take an external pointer named “**last**” that points to the last node of the list.

**last -> next** will point to the *first node*.



The pointer **last** points to node Z

**last -> next** points to node P.

### The idea of pointer that points to the last node instead of the first node

In General, insertion of a node at the beginning or at the end, the whole list has to be traversed.

If we take a pointer to the last node, then in both cases there won't be any need to traverse the whole list. (i.e. it takes constant time, irrespective of the length of the list)

# Algorithm

```
void circular_llist::add_begin(int val)
```

```
{
```

```
    node *temp = new Node();
```

```
    temp->info=val;
```

```
    if (last == NULL)
```

```
    {
```

```
        last=temp;
```

```
        temp->next=last;
```

```
    }
```

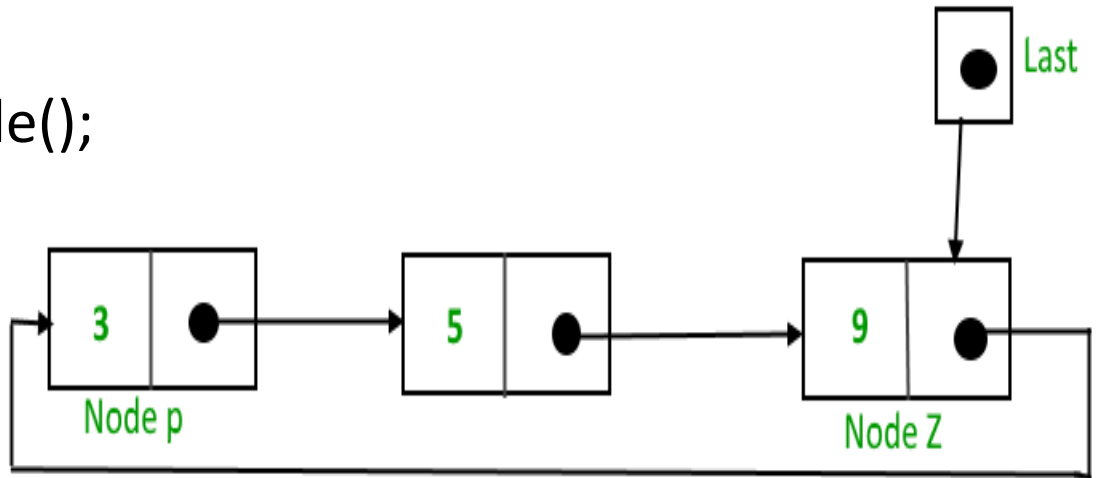
```
    else
```

```
    {
```

```
        temp->next = last->next;
```

```
        last->next = temp;
```

```
    }
```



# Algorithm Comparison

## Insert at Beginning

### Circular

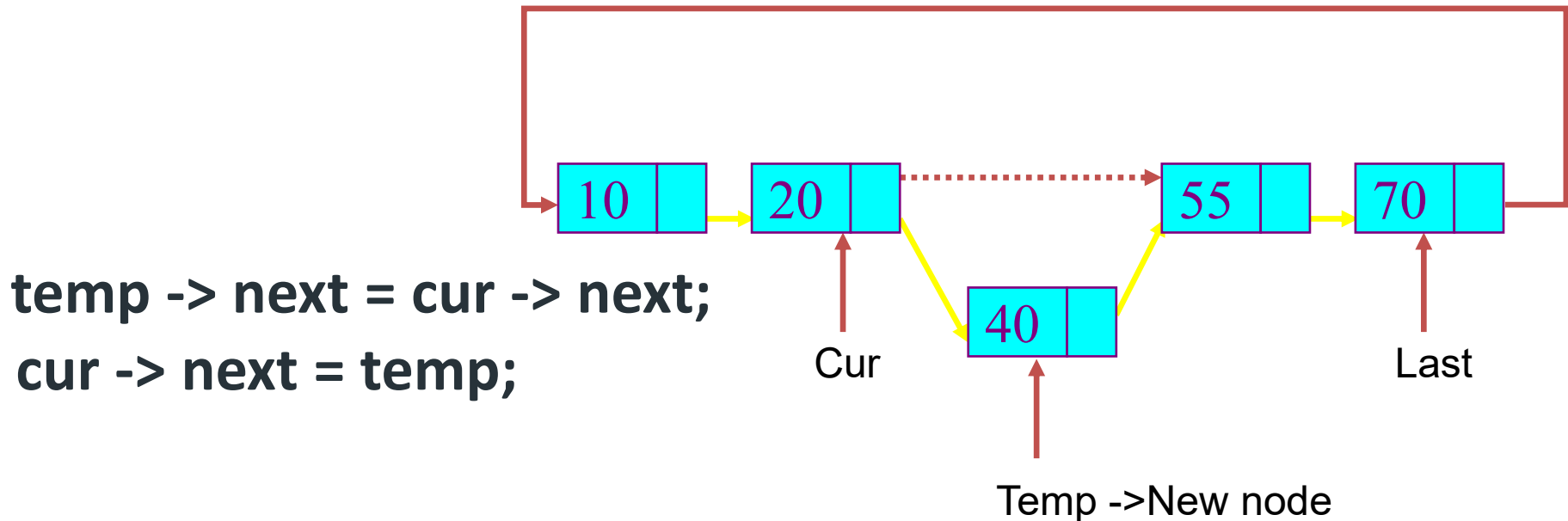
```
void circular_llist::add_begin(int val)
{
    node *temp = new Node();
    temp->info=val;
    if (last == NULL)
    {
        last=temp;
        temp->next=last;
    }
    else
    {
        temp->next = last->next;
        last->next = temp;
    }
}
```

### Single

```
void insert_beg(int val)
{ node *temp=new node;
  temp->info=val;
  If(head==NULL)
  {
      head=temp;
      temp->next=NULL
  }
  else{
      temp->next=head;
      head=temp; }
}
```

## Insertion between the Nodes of Circular Linked List

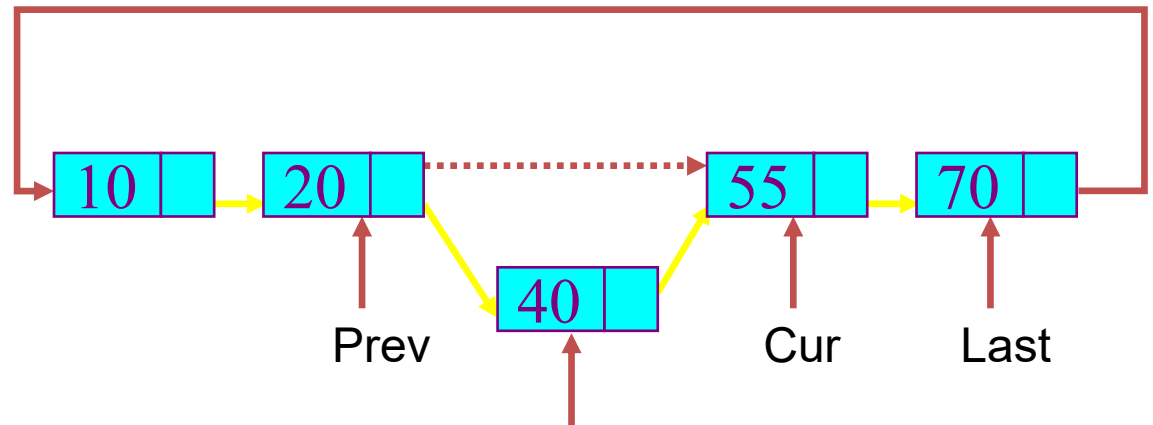
Search for the node after which new node to be inserted,  
(say node with value 20)



```
cur = last->next;  
While(cur-info != requiredVal)  
    cur = cur -> next;  
  
temp -> next = cur -> next;  
cur -> next = temp;
```

*Alternatively, we can also use two pointers ( prev & cur ) as follows*

```
temp -> next = cur;  
prev -> next = temp;
```



```
prev = last;  
cur = last->next;  
  
while(prev-info != requiredVal)  
    pre=cur  
    cur = cur -> next;  
  
temp -> next = cur;  
prev -> next = temp;
```

# Algorithm Comparison

## Insert at some position

### Circular list

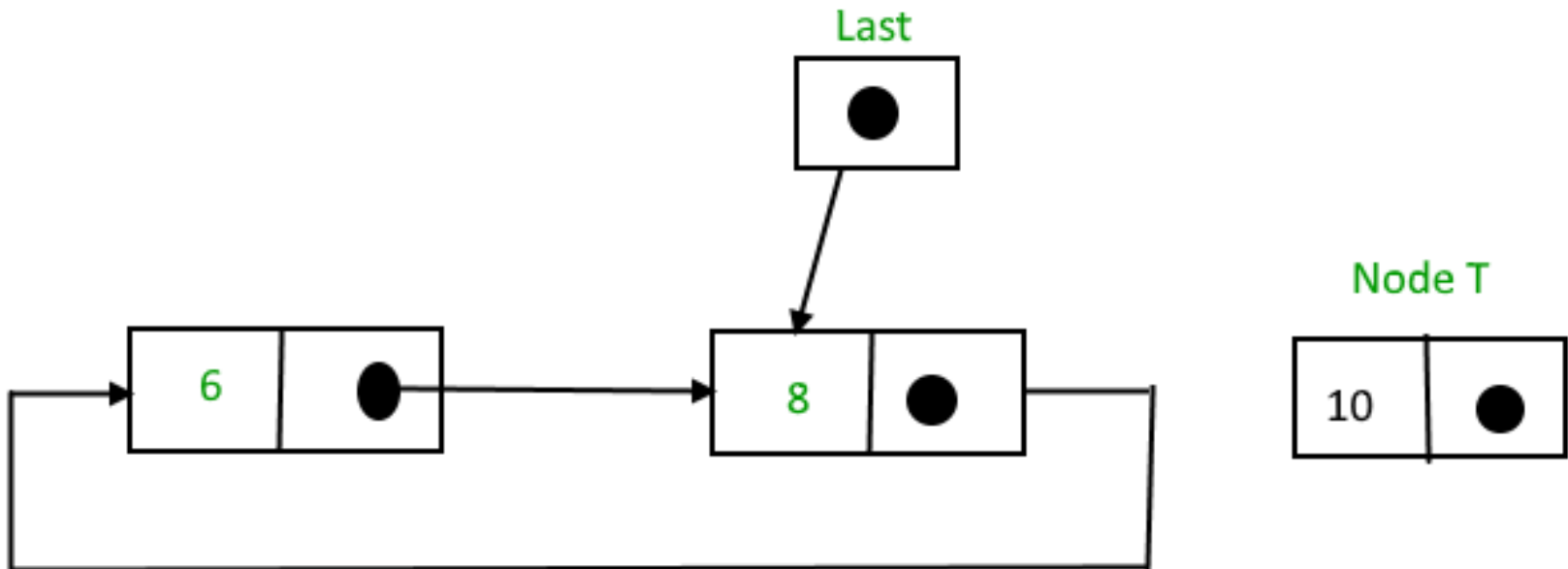
```
void insertNode( int item,int pos){
    Node *New = new Node();
    Node *Prev;
    Node *Cur;
    New->data = item;
    if(last == NULL)
    {
        last = New;
        last->next = last;
    }
    Prev = last;
    Cur = last->next;
    for (int i=1;i<pos;i++)
    {
        Prev = Cur;
        Cur = Cur->next;
    }
    New->next = Cur;
    Prev->next = New;
}
```

### Singly linear list

```
void insert_position(int pos, int val)
{
    node *temp=new node;
    node *pre;
    node *cur;
    temp->data=val;
    If(head==NULL)
    {
        head=temp;
        temp->next=NULL
    }
    cur=head;
    for(int i=1;i<pos;i++)
        { pre=cur; cur=cur->next; }
    temp->next=cur;
    pre->next=temp
}
```

## Insertion at the End of Circular Linked List

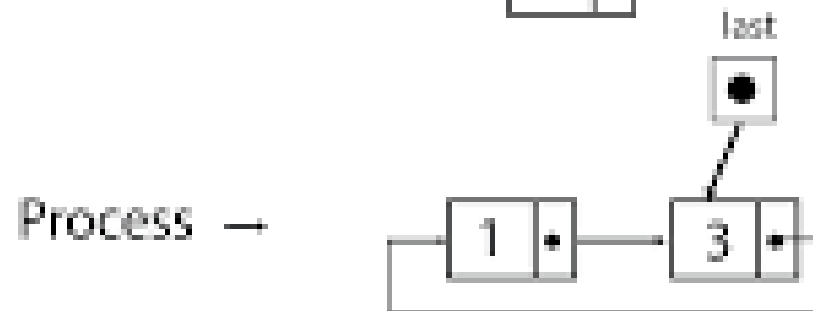
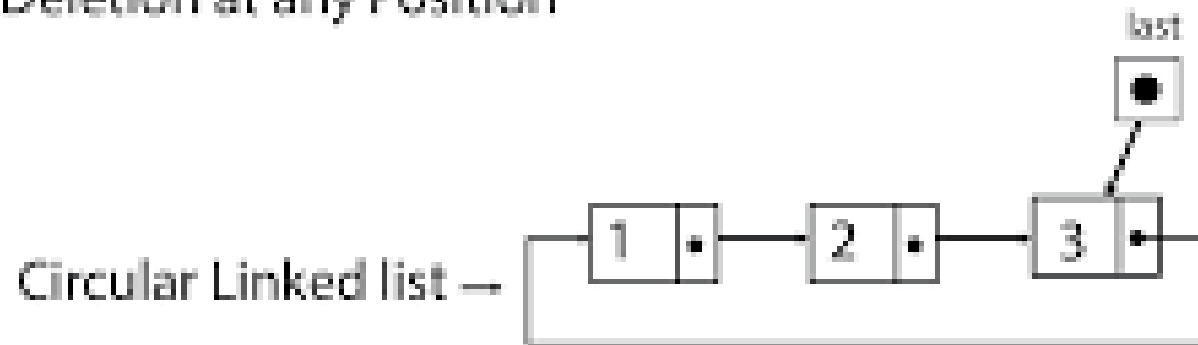
```
temp->next = last->next;  
last->next = temp;  
last = temp ;
```



# Deletion

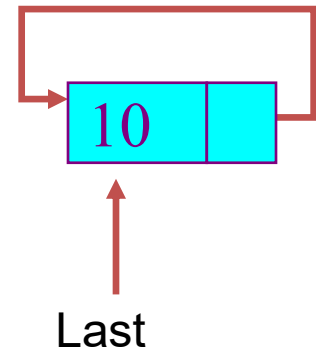
## Circular Linked List

Deletion at any Position



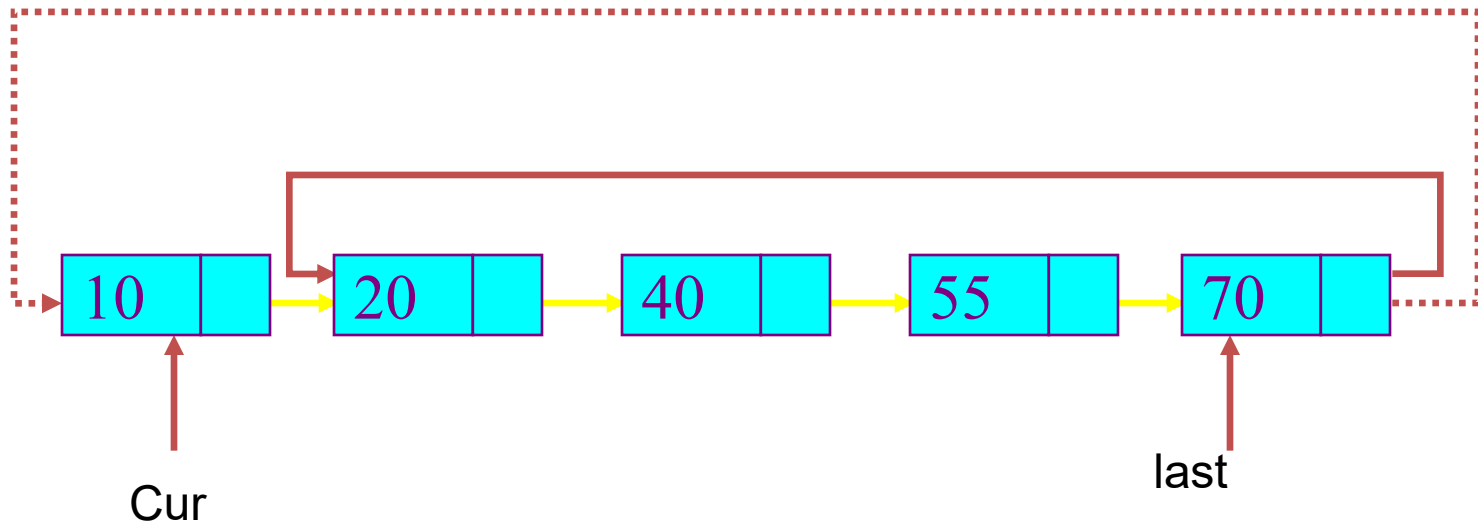
# Delete a node in a **single-node circular linked list**

```
if (last=last->next)  
{ cur = last;  
  last = NULL;  
  delete Cur; }
```



# Delete the head node from Circular List

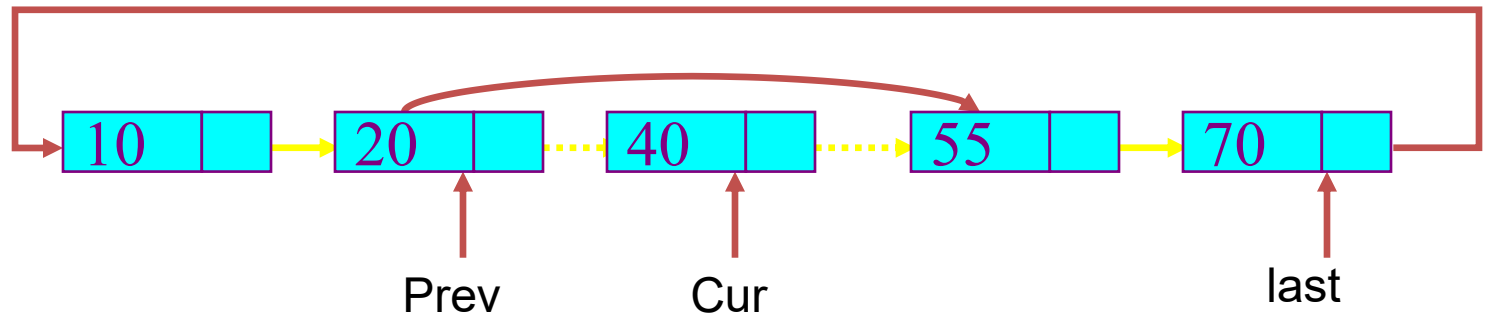
```
{ cur = last->next;  
  last->next = cur ->next;  
  delete Cur;  
}
```



# Delete a middle node from a Circular Linked List

Search for the node which is to be deleted,  
(say node with value 40)

**pre -> next = cur -> next;**  
**delete cur ;**



```
cur = last->next;  
pre = last  
While(cur-info != requiredVal)  
{pre= cur  
  cur = cur -> next;}
```

```
pre -> next = cur -> next;  
delete cur;
```

# Delete the end node from a Circular Linked List

```
cur = last->next
```

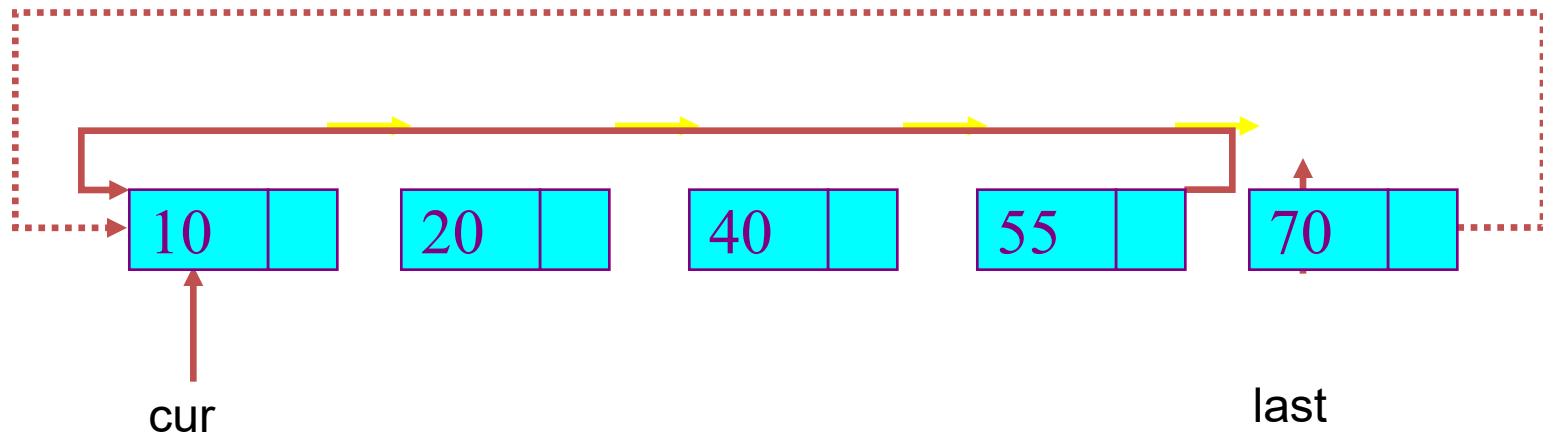
```
while (Cur->next!=last)
```

```
Cur=Cur->next;
```

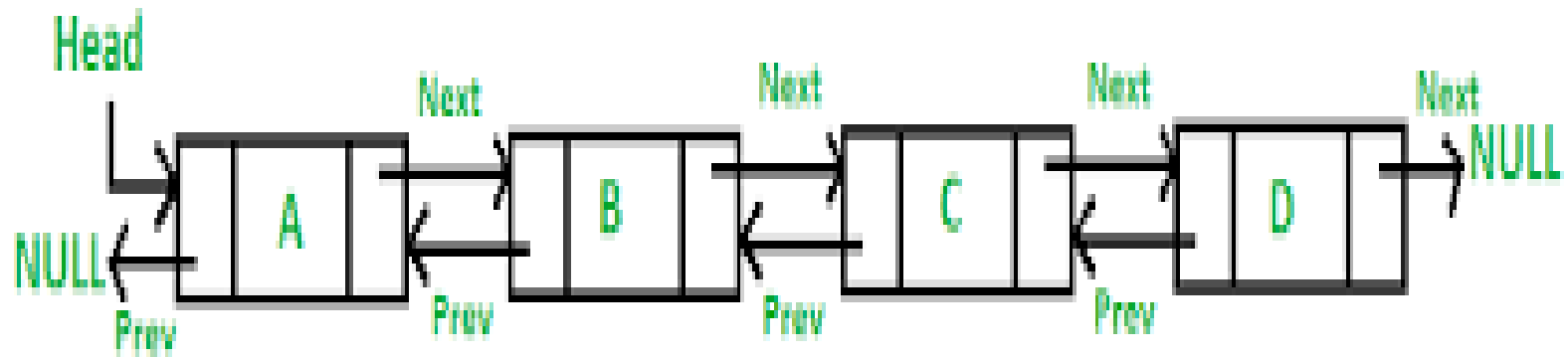
```
Cur-> next = last->next
```

```
delete last;
```

```
last = cur;
```



# Doubly Linked List

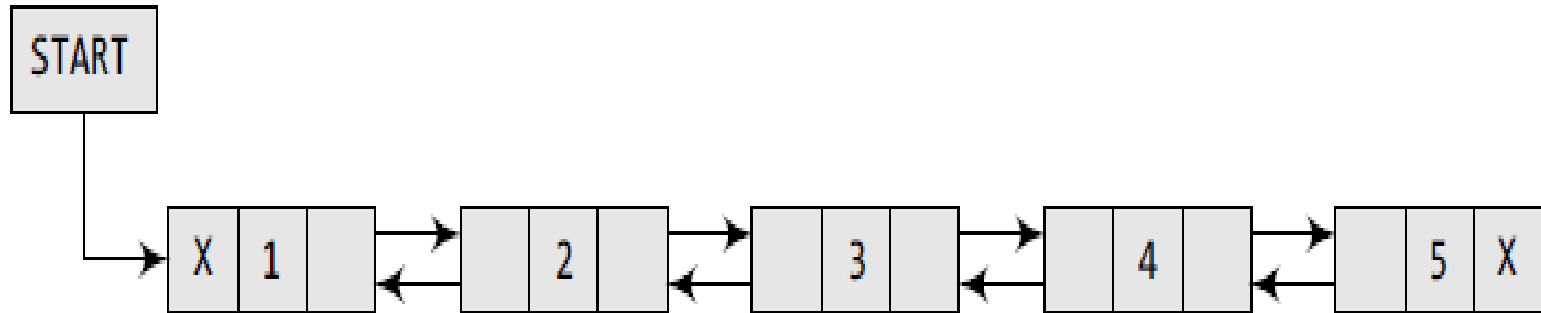


# Doubly Linked List

A doubly linked list or a two-way linked list is a more complex type of linked list which **contains a pointer to the next as well as the previous node** in the sequence.

Therefore, it consists of three parts—

1. data,
2. a pointer to the next node,
3. a pointer to the previous node



```
class Node
```

```
{
```

```
    int info;
```

```
    Node *next;
```

```
    Node *prev;
```

```
};
```

**PREV** field of the *first* node is **NULL**  
**NEXT** field of the *last* node is **NULL**.

The PREV field is used to store the address of the preceding node, which enables us to traverse the list in the backward direction.

# **Inserting a New Node in a Doubly Linked List**

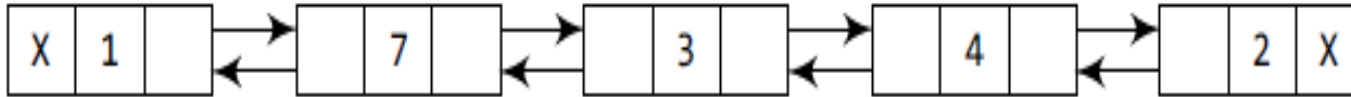
Case 1: The new node is inserted at the beginning.

Case 2: The new node is inserted at the end.

Case 3: The new node is inserted after a given node.

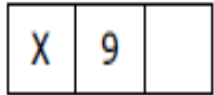
Case 4: The new node is inserted before a given node.

# Inserting a Node at the Beginning

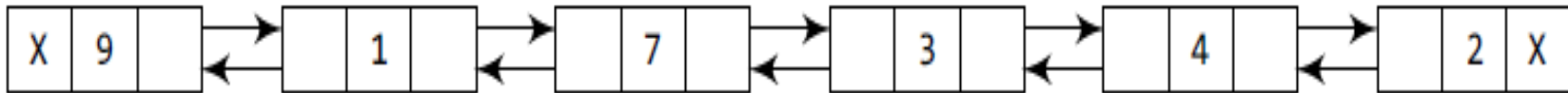


START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.



Add the new node before the START node. Now the new node becomes the first node of the list.



START

```
node *temp=new node;
```

```
temp->info=val; temp->prev=NULL;
```

```
If(start==NULL)
```

```
{ start=temp; temp->next=NULL;}
```

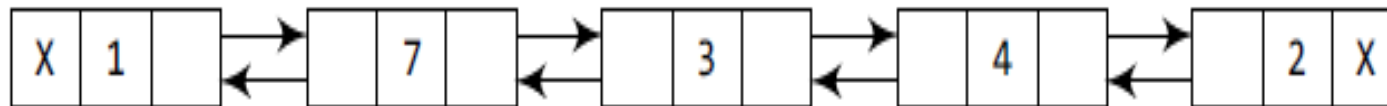
```
else{
```

```
temp->next=start;
```

```
start->prev=temp;
```

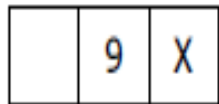
```
start=temp; }
```

# Inserting a Node at the End

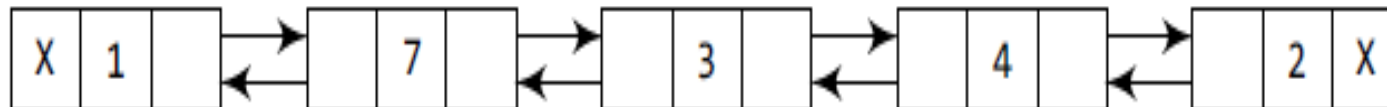


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

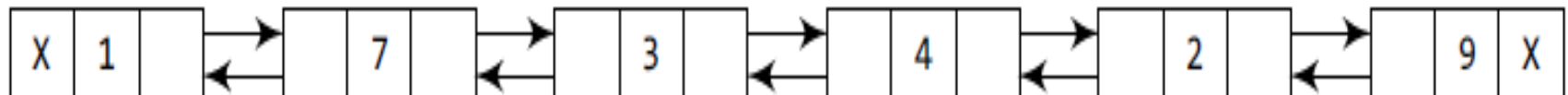


Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

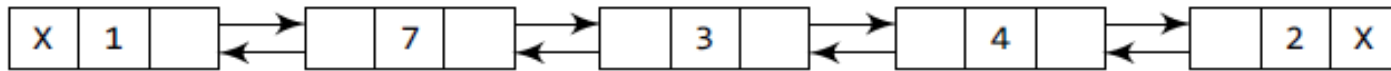
Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



START

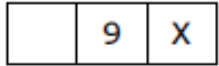
PTR

# Inserting a Node at the End

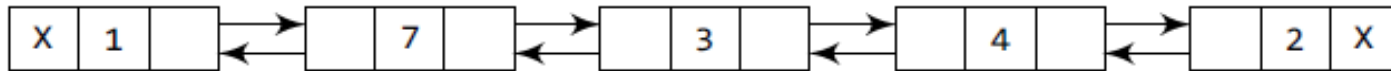


START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

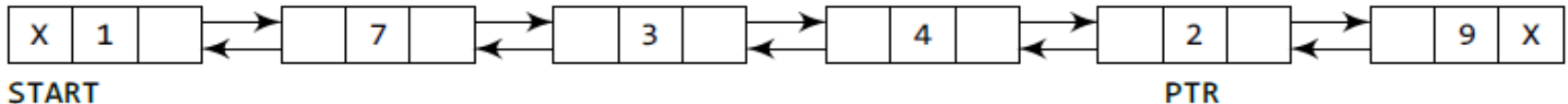


Take a pointer variable PTR and make it point to the first node of the list.



START, PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



START

PTR

```
node *temp=new node;
```

```
temp->info=val; temp->next=NULL;
```

```
Node *ptr = start;
```

```
while(ptr->next!=NULL)
```

```
{ ptr=ptr->next; }
```

```
ptr->next=temp;
```

```
temp->prev=ptr
```

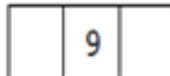
# Inserting a Node After a Given Node

*(say after third node)*

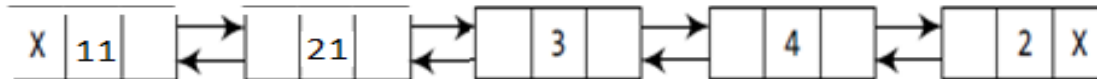


START

Allocate memory for the new node and initialize its DATA part to 9.



Take a pointer variable CUR make it point to the first node of the list.



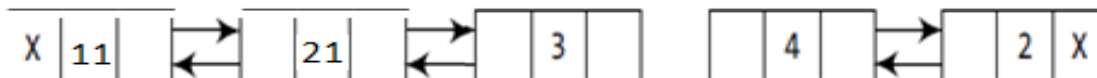
START CUR, PRE

Move CUR until the value after which the node has to be inserted.

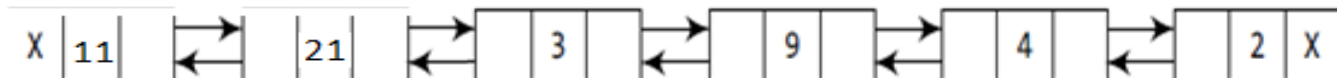
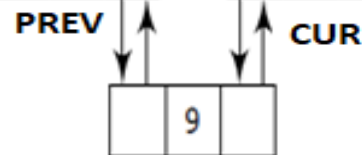


START

Insert the new node between

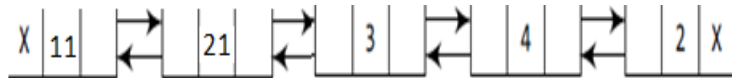


START



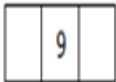
START

# Inserting a Node After a Given Node

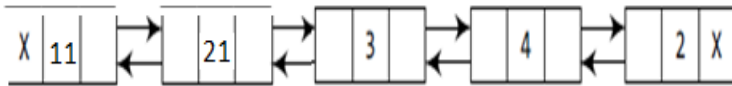


START

Allocate memory for the new node and initialize its DATA part to 9.

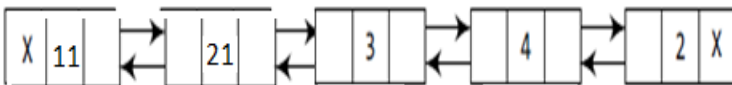


Take a pointer variable CUR make it point to the first node of the list.



START CUR, PRE

Move CUR until the value after which the node has to be inserted.

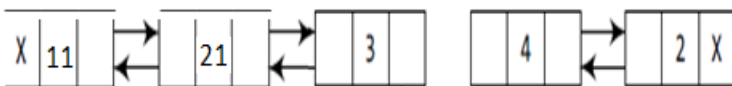


START

PREV

CUR

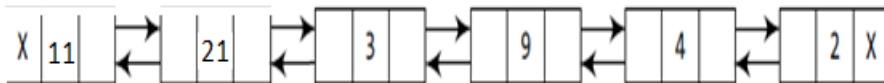
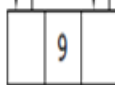
Insert the new node between



START

PREV

CUR



START

```
node *temp=new node;
```

```
temp->data=data;
```

```
pre= start
```

```
cur=start->next
```

```
while (pre->data!= sp_val)
```

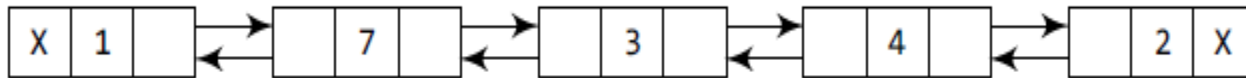
```
{ pre=cur; cur=cur->next; }
```

```
temp->next=cur; temp->prev=pre;
```

```
pre->next=temp;
```

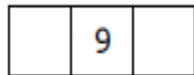
```
cur->prev==temp;
```

# Inserting a Node Before a Given Node in a Doubly Linked List

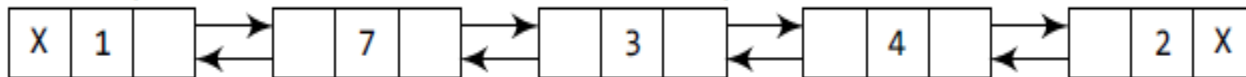


START

Allocate memory for the new node and initialize its DATA part to 9.

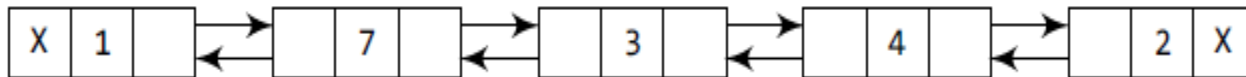


Take a pointer variable PTR and make it point to the first node of the list.



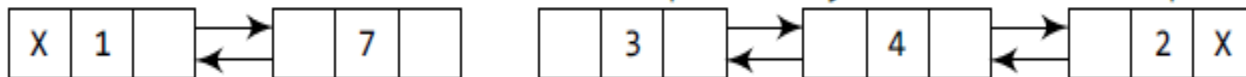
START, PTR

Move PTR further so that it now points to the node whose data is equal to the value before which the node has to be inserted.

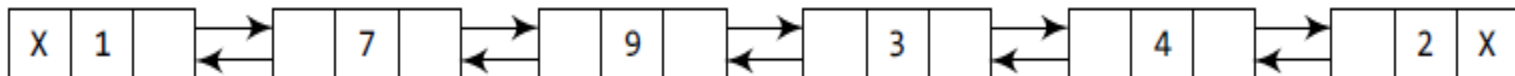
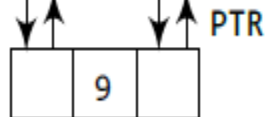


START

Add the new node in between the node pointed by PTR and the node preceding it.

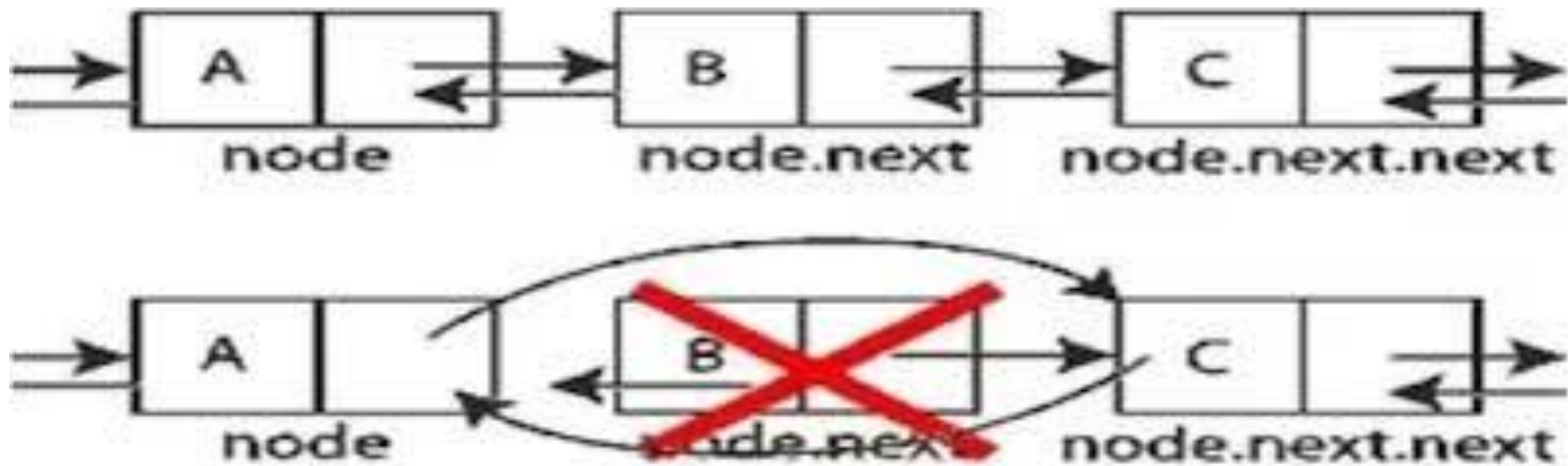


START



START

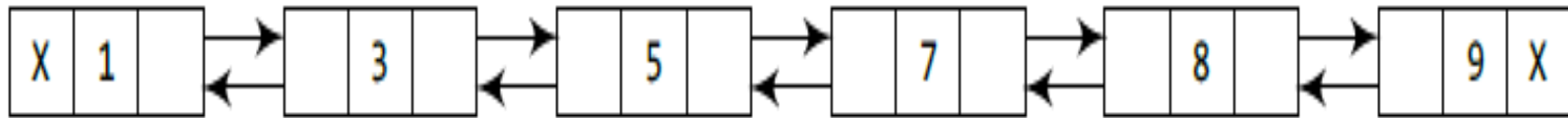
# Deleting a Node from a Doubly Linked List



## **Deleting a Node from a Doubly Linked List**

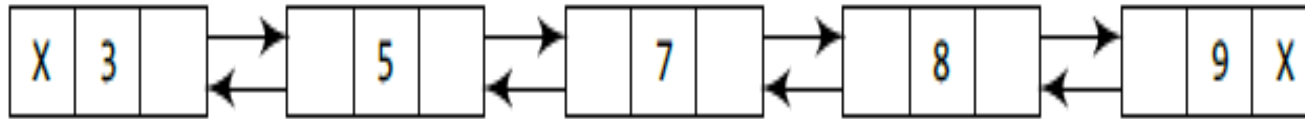
- Case 1: The first node is deleted.
- Case 2: The last node is deleted.
- Case 3: The node after a given node is deleted.
- Case 4: The node before a given node is deleted

# Deleting the First Node



START

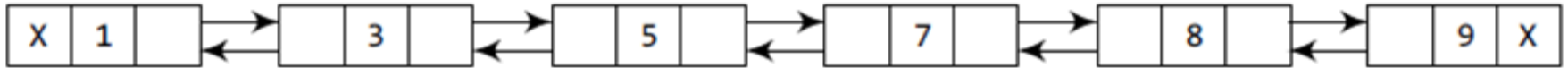
Free the memory occupied by the first node of the list and make the second node of the list as the START node.



START

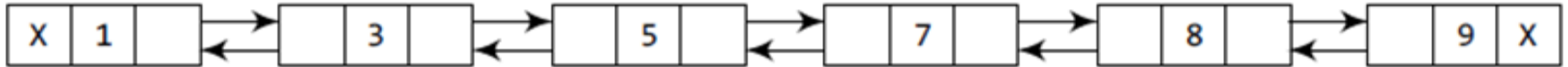
```
if (start == NULL)
    cout << "Underflow" << endl;
else
    node *ptr;
    ptr = head;
    head = head->next;
    head->prev = NULL;
    delete ptr;
```

# Deleting the Last Node



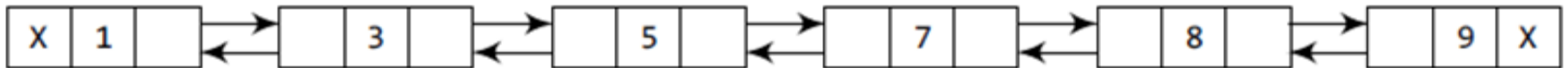
START

Take a pointer CUR, PRE that points to the first node of the list.



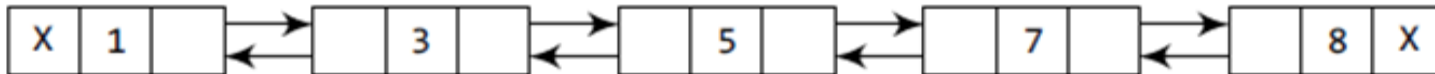
START, CUR, PRE

Move CUR, so that it now points to the last node of the list.



START

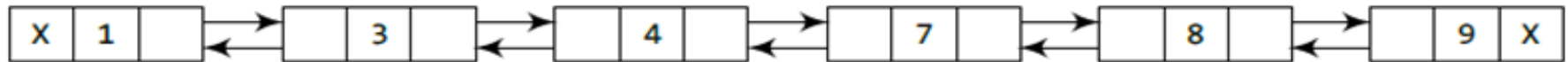
Free the space occupied by the node pointed by CUR and store NULL in NEXT field of its preceding node.



START

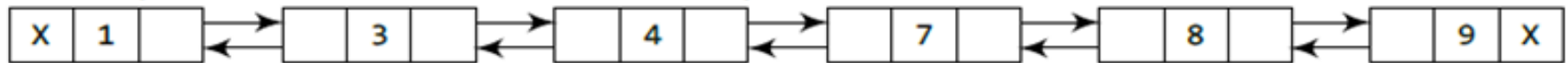
```
while (CUR->next != NULL)
{
    PRE = CUR;
    CUR = CUR->next;
}
PRE->next = NULL;
delete CUR;
```

# Deleting the Node After a Given Node



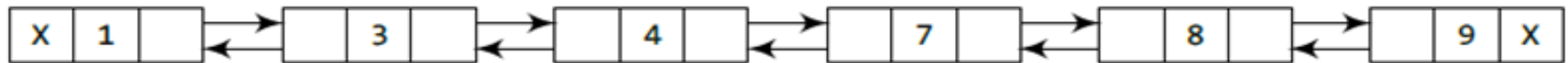
START

Take a pointer variable PTR and make it point to the first node of the list.



START, CUR PRE

Move CUR further so that its data part is equal to the value after which the node has to be inserted.



START

Delete the node succeeding PRE.



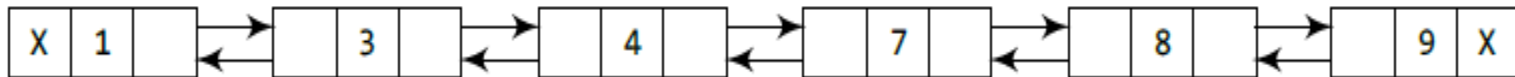
START



START

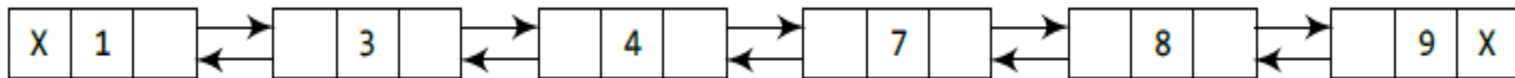
```
cur=start;
while (CUR->data!= sp_val)
    { PRE=CUR; CUR=CUR->next; }
PRE->next=CUR->next;
(CUR->next)->pre=PRE;
delete CUR;
```

# Deleting the Node Before a Given Node in a Doubly Linked List



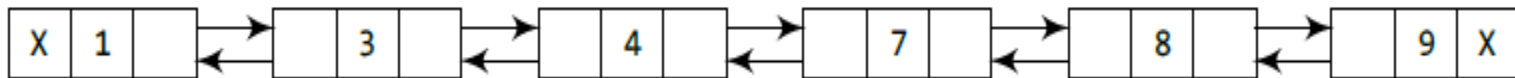
START

Take a pointer variable PTR that points to the first node of the list.



START, PTR

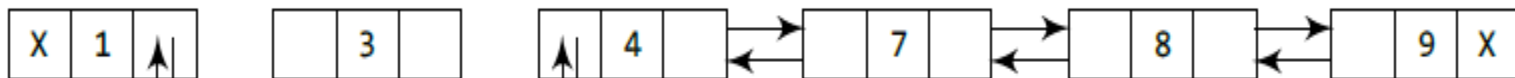
Move PTR further till its data part is equal to the value before which the node has to be deleted.



START

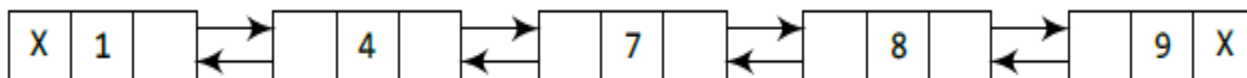
PTR

Delete the node preceding PTR.



START

PTR



START

## Practice Activity

- Implement Doubly Linked list by writing functions for all possible operations in doubly linked list.
- Reverse a singly Linked List

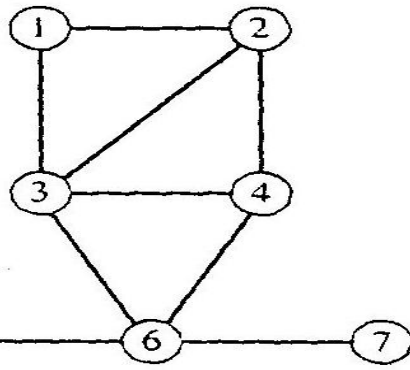
## Applications of Linked List

- To implement many data structure for dynamic allocation like Stacks, Queues etc.
- To implement the Adjacency list representation of Graph.
- To implement Hash Tables – Ex. Open Chain Hashing (Each Bucket of the hash table can itself be a linked list.
- To represent a polynomial by simply storing the coefficient and exponent of each term.
- To implement “Undo” functionality in many software such as Photoshop or Word by storing Linked list of states.
- Operating System maintains and control all the running applications by creating circular linked list
- Representing Sparse Matrix

# Adjacency Lists Representation

- From

– to

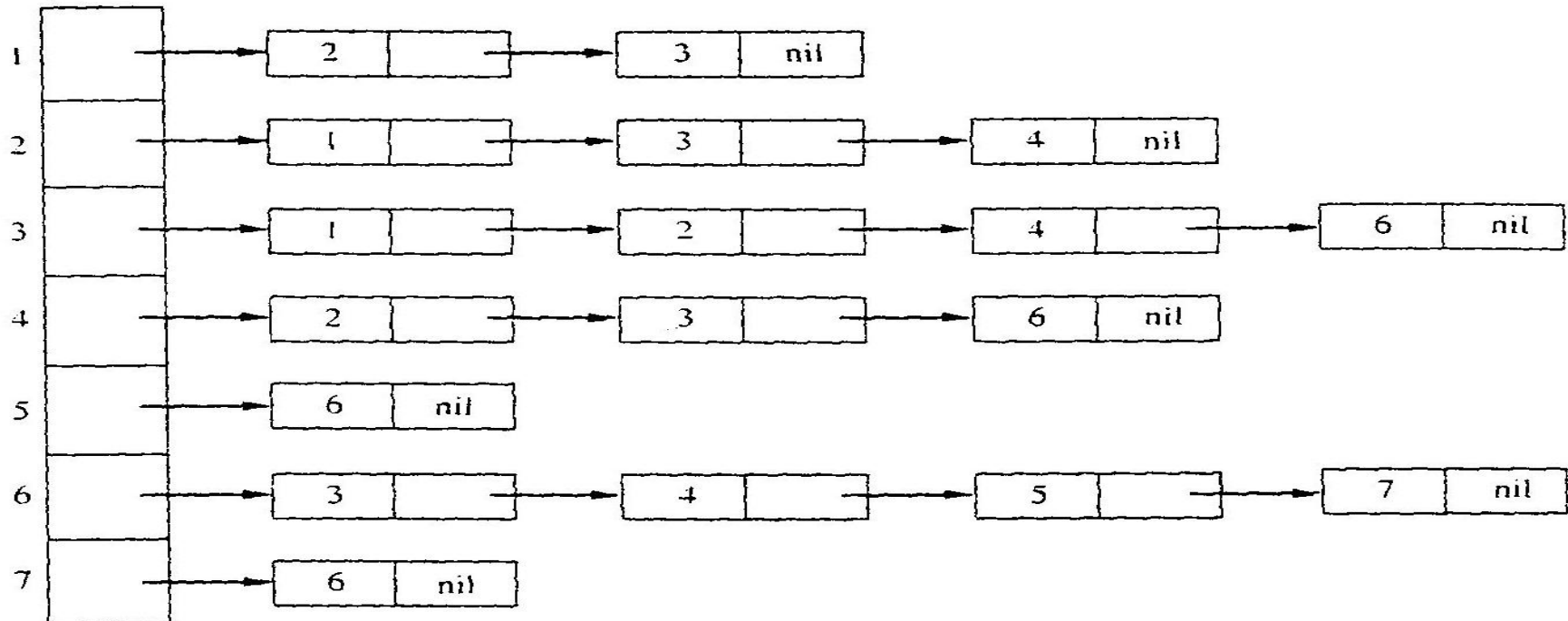


(a) An undirected graph

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(b) Its adjacency matrix

adjVertices



## Complexity Analysis of Linked List Operations

| List Type | Time Complexity |           |          |
|-----------|-----------------|-----------|----------|
|           | Worst           |           |          |
|           | Search          | Insertion | Deletion |
|           |                 |           |          |

Singly-Linked List

Doubly-Linked List

Circular Linked List

## Complexity Analysis of Linked List Operations

### Worst Time Complexity

| List Type                   | Search | Insertion |        |        | Deletion |        |        |
|-----------------------------|--------|-----------|--------|--------|----------|--------|--------|
|                             |        | Beg       | Btw    | End    | Beg      | Btw    | End    |
| <i>Singly-Linked List</i>   | $O(n)$ | $O(1)$    | $O(n)$ | $O(n)$ | $O(1)$   | $O(n)$ | $O(n)$ |
| <i>Doubly-Linked List</i>   | $O(n)$ | $O(1)$    | $O(n)$ | $O(n)$ | $O(1)$   | $O(n)$ | $O(n)$ |
| <i>Circular Linked List</i> | $O(n)$ | $O(1)$    | $O(n)$ | $O(1)$ | $O(1)$   | $O(n)$ | $O(n)$ |